

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

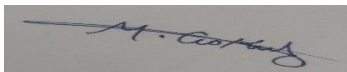
Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with	BL3 – Apply	5

	integrity constraint enforcement for a given scenario.		
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

*I **M.GOKUL(192565067)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with	Mostly correct steps with minor	Disorganized steps; multiple errors	No clear process

		correct approach throughout	mistakes		
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

Employee (Emp ID, Name, Salary, Dept ID)

Emp ID	Name	Salary	Dept ID
101	Rahul	60000	D1
102	Priya	45000	D2
103	Arun	70000	D1

Department (Dept ID, Dept Name)

Dept ID	Dept Name
D1	IT
D2	HR

1. Selection (σ)

Retrieve employees earning more than ₹50,000.

Relational Algebra

σ Salary > 50000(Employee)

Output

Rahul
Arun

2. Projection (π)

Retrieve only employee names.

π Name (Employee)

Output

Rahul
Priya
Arun

3. Union ($\cup$)

Suppose another relation

New Employee (Emp ID, Name)

Union:

π Name (Employee) $\cup$ π Name (New Employee)

Returns all employee names without duplicates.

4. Set Difference ($-$)

Employees not in HR.

π Emp ID(Employee)

$-$

π Emp ID (Employee $\bowtie$ Department where Dept Name='HR')

Output

101

5. Cartesian Product (×)

Employee × Department

Every employee is combined with every department.

If Employee has 3 rows and Department has 2 rows

Result = $3 \times 2 = 6$ tuples.

6. Join (⋈)

Employee ⋈ Employee. Dept ID = Department . Dept ID Department

Output

Name Dept Name

Rahul IT

Priya HR

Arun IT

2. Write SQL DDL statements to create STUDENT (SID, Name, DOB, DeptID) and DEPARTMENT(Dept ID, Dept Name) tables with all appropriate constraints.

```
CREATE TABLE Department(
DeptID INT PRIMARY KEY,
DeptName VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE Student(
SID INT PRIMARY KEY,
Name VARCHAR(50) NOT NULL,
DOB DATE,
DeptID INT,
FOREIGN KEY (DeptID)
REFERENCES Department(DeptID)
);
```

Constraints Used

- PRIMARY KEY ensures uniqueness.
- NOT NULL prevents empty values.
- FOREIGN KEY maintains referential integrity.
- DATE datatype stores date of birth.

3.SQL Queries Using GROUP BY and Aggregate Functions

Assume

Sales(SaleID, Product, Amount)

COUNT

```
SELECT COUNT(*) AS TotalSales  
FROM Sales;
```

Sum

```
SELECT SUM(Amount)  
FROM Sales;
```

Average

```
SELECT AVG(Amount)  
FROM Sales;
```

Maximum

```
SELECT MAX(Amount)  
FROM Sales;
```

Minimum

```
SELECT MIN(Amount)  
FROM Sales;
```

GROUP BY

```
SELECT Product,  
SUM(Amount)  
FROM Sales  
GROUP BY Product;
```

HAVING

```
SELECT Product,  
SUM(Amount)  
FROM Sales  
GROUP BY Product  
HAVING SUM(Amount)>10000;
```

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

Find employees earning more than the average salary of their department

```
SELECT Name,  
Salary,  
DeptID  
FROM Employee E  
WHERE Salary >  
(  
SELECT AVG(Salary)  
FROM Employee  
WHERE DeptID=E.DeptID  
);
```

Explanation

The inner query calculates the average salary of the employee's own department.

The outer query compares each employee's salary with that average.

Q5. Apply All Types of JOIN

INNER JOIN

```
SELECT *  
FROM Employee E  
INNER JOIN Project P  
ON E.EmpID=P.EmpID;
```

LEFT OUTER JOIN

```
SELECT *  
FROM Employee E  
LEFT JOIN Project P  
ON E.EmpID=P.EmpID;
```

RIGHT OUTER JOIN

```
SELECT *  
FROM Employee E
```

```
RIGHT JOIN Project P
ON E.EmpID=P.EmpID;
FULL OUTER JOIN
SELECT *
FROM Employee E
FULL OUTER JOIN Project P
ON E.EmpID=P.EmpID;
```

```
CROSS JOIN
SELECT *
FROM Employee
CROSS JOIN Project;
```

Q6. Create View

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
COUNT(E.EmpID) AS TotalEmployees,
AVG(E.Salary) AS AvgSalary
FROM Employee E
JOIN Department D
ON E.DeptID=D.DeptID
GROUP BY D.DeptName;
```

Query on View

```
SELECT *
FROM DeptSalaryView;
```

Advantages

- Security
- Simplicity
- Reusability
- Data abstraction

Q7. Relational Algebra Expression

Relations

Student(SID, Name)

Course(CID)

Enrollment(SID, CID)

Find students enrolled in every course.

Expression

π Name

(Student $\bowtie$

(

π SID,CID(Enrollment)

$\div$

π CID(Course)

))

Explanation

Division ($\div$) finds students who are enrolled in all available courses.

Q8. SQL DML Statements

INSERT

INSERT INTO Student

VALUES(101,'Rahul','2004-05-20',1);

UPDATE

UPDATE Student

SET Name='Rohit'

WHERE SID=101;

DELETE

DELETE FROM Student

WHERE SID=101;

Integrity Constraints

- Primary Key
- Foreign Key
- NOT NULL
- UNIQUE

These constraints ensure data consistency and prevent invalid data.

Q9. Tuple Relational Calculus Expression

$$\{ E.Name \mid$$
$$Employee(E)$$
$$\wedge$$
$$Department(D)$$
$$\wedge$$
$$E.DeptID=D.DeptID$$
$$\wedge$$
$$D.DeptName='IT'$$
$$\wedge$$
$$E.Salary>50000$$
$$\}$$

Explanation

The expression retrieves employee names satisfying both conditions:

- Department = IT
- Salary greater than ₹50,000.

Q10. Airline Reservation System

Relational Schema

Passenger

(PassengerID, Name, Phone)

Flight

(FlightID, FlightName, Source, Destination)

Booking

(BookingID, PassengerID, FlightID, SeatNo)

Payment

(PaymentID, BookingID, Amount)

SQL Query 1 (JOIN)

```
SELECT P.Name,  
F.FlightName  
FROM Passenger P  
JOIN Booking B  
ON P.PassengerID=B.PassengerID  
JOIN Flight F  
ON B.FlightID=F.FlightID;
```

SQL Query 2 (Subquery)

```
SELECT Name  
FROM Passenger  
WHERE PassengerID IN  
(  
SELECT PassengerID  
FROM Booking  
);
```

SQL Query 3 (Aggregate)

```
SELECT FlightID,  
COUNT(*) AS TotalBookings  
FROM Booking  
GROUP BY FlightID;
```

SQL Query 4 (View)

```
CREATE VIEW FlightDetails AS  
SELECT P.Name,  
F.FlightName,
```

```
B.SeatNo  
FROM Passenger P  
JOIN Booking B  
ON P.PassengerID=B.PassengerID  
JOIN Flight F  
ON B.FlightID=F.FlightID;
```

SQL Query 5 (Using View)

```
SELECT *  
FROM FlightDetails;
```